

STM Creator Operation Manual

This manual explains how to use STM Creator from a user and administrator perspective. It covers the deployed system, local startup, account flows, model editing, collaboration, permissions, review locks, milestones, import/export, and common troubleshooting steps.

Production deployment:

- Frontend editor: <https://stm-8nizc.ondigitalocean.app/editor>
- Backend API used by the frontend source: <https://hammerhead-app-t8l9y.ondigitalocean.app>

1. System Purpose

STM Creator is a browser-based State-and-Transition Model editor. It allows users to:

- Create or select an ecological STM model.
- Add, edit, duplicate, and delete state nodes.
- Add, edit, and delete transitions between states.
- Record transition drivers, preconditions, causal chains, and calculated deltas.
- Save models to a PostgreSQL/PostGIS backend.
- Import and export EKS-style JSON.
- Export the current graph as a PNG.
- Collaborate with other users through presence, cursors, comments, entity locks, and milestones.
- Manage user and model permissions when the current user has the required role.

The frontend is a React/Vite application. The backend is an Express/TypeScript API with PostgreSQL, Socket.IO, JWT authentication, and database migrations.

2. User Roles

STM Creator uses two permission layers: global roles and model-specific roles.

Global Roles

Role	Typical Scope
Admin	Full system administration. Can access the admin dashboard, inspect users, update global roles, remove sessions, and apply review locks.
Editor	Normal authenticated editing user. Can create models and edit models where model permission allows it.
Viewer	Authenticated user with read-only default access. A new signup is created as Viewer until an administrator or owner grants more access.

Important behavior:

- New accounts are not automatically elevated to **Editor**.

- If a user can log in but cannot create or save models, check both global role and model role.
- The admin dashboard is protected on both frontend and backend. Client-side hiding is not the security boundary.

Model Roles

Model Role	Capabilities
owner	Full control of the model. Can edit, save, delete, and manage model permissions.
editor	Can edit and save the model.
viewer	Can open and inspect the model but cannot save editing changes.

Model permissions are assigned per model. A user may be a viewer on one model and editor or owner on another.

3. Accessing the Deployed System

Open the production editor in a browser:

```
https://stm-8nizc.ondigitalocean.app/editor
```

Recommended browsers:

- Microsoft Edge
- Google Chrome
- Other current Chromium-based browsers

The application is a browser app, so no desktop installation is required. The deployed frontend talks to the backend API configured at build time.

If the app loads but authentication or model requests fail, check:

- Whether the backend deployment is online.
- Whether browser developer tools show CORS errors.
- Whether the frontend was built with the intended `VITE_API_BASE_URL`.
- Whether the current user has a valid verified account.

4. Local Startup

Local startup is useful for development, demonstrations, and debugging.

4.1 Prerequisites

Install:

- Node.js compatible with the project lockfiles.
- npm.
- PostgreSQL with PostGIS, or Docker Desktop for the compose setup.

- Git.

The repository contains two separate applications:

```
stm_creator-frontend/  
tern_backend/
```

4.2 Start the Backend Locally

Open a terminal in `tern_backend` :

```
cd tern_backend  
npm install
```

Create a `.env` file from `.env.example` and provide real values. Common variables include:

```
NODE_ENV=development  
PORT=3000  
DATABASE_URL=postgres://postgres:postgres@localhost:5432/tern  
JWT_SECRET=replace-with-a-long-secret  
PASSWORD_PEPPER=replace-with-a-long-secret  
FRONTEND_URL=http://localhost:5173
```

Run migrations:

```
npm run migrate:up
```

Start the backend:

```
npm run dev
```

Expected local backend URL:

```
http://localhost:3000
```

Useful backend endpoints:

- `GET /auth/health`
- `GET /models/health`
- `GET /api-docs` for Swagger UI when enabled by the app.

4.3 Start Backend Dependencies with Docker

The backend includes `docker-compose.yml` for local PostGIS and app services.

From `tern_backend` :

```
docker compose up --build
```

If you only need the database container, adapt the compose command to the service names in `docker-compose.yml`.

4.4 Start the Frontend Locally

Open a second terminal in `stm_creator-frontend`:

```
cd stm_creator-frontend
npm install
```

Set the local backend base URL in `.env.development`:

```
VITE_API_BASE_URL=http://localhost:3000
```

Start Vite:

```
npm run dev
```

Expected local frontend URL:

```
http://localhost:5173
```

5. Login and Accounts

5.1 Signup

Use the signup form on the authentication page.

Typical required fields:

- Name.
- Email.
- Password.

After signup:

- The backend creates a user.
- The default global role is usually `Viewer`.
- The backend creates an email verification token.
- A verification email is sent if email settings are configured.

5.2 Email Verification

Email verification is required for normal authenticated API access.

Verification flow:

1. User receives a verification email.
2. User opens the verification link.
3. Frontend route consumes the token.

4. Backend marks the account as verified.
5. User can log in normally.

If verification email is not delivered, check backend mail environment variables and email service logs.

5.3 Login

Use email and password on the login page.

Successful login stores a JWT in the browser. The frontend then sends:

```
Authorization: Bearer <token>
```

for authenticated API calls and Socket.IO connection authentication.

5.4 Guest Mode

The frontend contains a guest entry path for trying the editor. Guest mode should be treated as limited and is not a substitute for a verified account when persistent model saves, permissions, and collaboration are required.

6. Home Page and Routes

Important frontend routes:

Route	Purpose
/	Home page.
/editor	Main STM editor. Production entry: <code>https://stm-8nizc.ondigitalocean.app/editor</code> .
/verify-email	Email verification result page.
/admin	Admin dashboard. Requires <code>Admin</code> .
/forbidden	Access denied page.
*	Not found page.

The editor is the main working surface. Users normally authenticate first, then choose or create a model.

7. Select or Create a Model

After login, the app shows model selection options.

7.1 Existing Models

The model list may include:

- Models owned by the user.
- Models explicitly shared with the user.

- Templates available for cloning.
- Models visible because the user is an administrator.

Open a model to load its states, transitions, drivers, node positions, comments, milestones, and permission context.

7.2 Create a Blank Model

If the user has sufficient global permission, they can create a blank model.

Recommended naming rules:

- Use a clear ecosystem or project name.
- Avoid punctuation that could be confusing in URLs.
- Do not reuse protected template names.

7.3 Create from Template

Default templates include:

- Woodlands
- Grasslands
- Shrublands

Template models are protected. Users should clone a template instead of editing the template directly.

8. Editor Interface

The editor screen is made of:

- Top toolbar.
- React Flow canvas.
- State nodes.
- Transition edges.
- Right-side comment panel.
- Modals for nodes, transitions, versions, sharing, model lists, and help.
- Collaboration indicators such as online users, cursors, and locks.

The toolbar is grouped into menus.

8.1 Canvas Menu

Typical canvas actions:

- Add a state node.
- Enable transition creation mode.
- Fit view.
- Reset or change layout.
- Access canvas-level utilities.

8.2 Model Menu

Typical model actions:

- Select or switch model.
- Create a model.
- Clone from template.
- Save model.
- Delete model.
- Manage model permissions.
- Manage milestones.

8.3 File Menu

Typical file actions:

- Import EKS JSON.
- Export EKS JSON.
- Export PNG.

8.4 View Menu

Typical view actions:

- Show or hide comments.
- Show transition filters.
- Open help.
- Toggle auxiliary editor panels.

The exact menu labels may vary as the frontend evolves, but the above groups match the current code organization.

9. State Node Operations

9.1 Create a State Node

Common ways to create a state:

- Use the toolbar action.
- Use a canvas context menu if enabled.

The node modal captures state data such as:

- State ID or title.
- State description.
- VAST class or condition class.
- Condition range.
- Notes.
- Optional image or reference metadata.
- Template-based defaults.

After creation, the state appears as a node on the canvas. Its position is saved with the model when backend save succeeds.

9.2 Edit a State Node

Open a state by clicking or double-clicking the node depending on the active editor mode.

Editable information includes:

- Display title.
- Description.
- Condition class.
- Condition range.
- Notes.
- Image-related metadata.

When collaboration is active, another user may temporarily lock the entity. If a lock is held by someone else, the UI should prevent conflicting edits.

9.3 Duplicate a State Node

If enabled in the current context menu, duplication creates a similar state with a new identifier. Review the new state carefully before saving, especially if transitions should not be copied.

9.4 Delete a State Node

Deleting a state can also affect transitions connected to that state.

Before deleting:

- Confirm that the state is not needed by another workflow.
- Check incoming and outgoing transitions.
- Save or create a milestone if the deletion is significant.

Deletion may be blocked if the user does not have edit permission or if the model is under review lock.

10. Transition Operations

10.1 Create a Transition

Transitions connect one state to another.

Typical flow:

1. Enable transition creation mode.
2. Select the source state.
3. Select the target state.
4. Fill in transition details in the modal.
5. Save the transition.

10.2 Edit Transition Basic Information

The transition modal supports fields such as:

- Transition identifier.
- Name or title.
- Description.
- Source and target state context.
- Driver selection.
- Precondition.
- Metrics needed for delta calculation.

10.3 Delta Calculation

The frontend and backend share transition delta logic. The exact helper names differ by layer, but the intent is consistent: calculate a transition change value from available transition data.

If a delta shown in the UI differs from the saved backend result, check:

- Whether the transition fields were fully saved.
- Whether frontend and backend helper logic are aligned.
- Whether imported JSON contains older field names.

10.4 Edit Causal Chains

A transition may contain causal chain information, including:

- Driver.
- Sub-driver.
- Precondition.
- Intermediate causal notes.
- Description of transition mechanism.

Driver search is backed by the backend `/drivers` endpoints.

10.5 Delete a Transition

Use the transition context menu or modal action when available.

Deletion may be blocked when:

- The current user is not an editor or owner.
- The model is locked for review.
- The transition is currently locked by another collaborator.
- The backend cannot resolve the transition business ID.

11. Save Model

Saving sends the current model graph to the backend.

The save payload includes:

- Model metadata.

- States.
- State positions.
- Transitions.
- Drivers and sub-drivers.
- Template flag where relevant.

Successful save updates the backend database and should preserve the model owner and permissions.

Common save blockers:

- User is unauthenticated.
- User email is not verified.
- User is only a `viewer` for the model.
- User has global `Viewer` role and no model edit grant.
- Model has an active review lock.
- Required fields are missing.
- Backend validation rejects the payload.
- Backend database is unavailable.
- Frontend is pointing at the wrong backend base URL.

Recommended save practice:

- Save after a focused group of edits.
- Create a milestone before risky structural changes.
- Refresh the model after resolving a failed save to avoid overwriting newer server state.

12. Comments and Mentions

The comment panel supports model discussion and review notes.

Typical actions:

- Add a comment.
- Mention another user by email or supported mention syntax.
- Resolve a comment.
- Delete a comment where permission allows it.
- Read recent comments for the current model.

Mentions can create notification records. Users can retrieve and mark mention notifications as read through the notifications API.

Comment failures are usually caused by:

- Missing authentication.
- Missing model permission.
- Empty comment body.
- Backend validation errors.
- Network or CORS problems.

13. Real-Time Collaboration

The editor uses Socket.IO for live collaboration.

Features include:

- Join a model room.
- Show online users.
- Broadcast cursor movement.
- Broadcast viewport updates.
- Acquire and release entity locks.
- Broadcast entity patches.
- Receive new comments and milestones.
- Receive model restore notifications.

Collaboration requires a valid JWT. The socket handshake uses the same authentication concept as the REST API.

If collaboration does not work:

- Confirm the user is logged in.
- Confirm the token is not expired.
- Check browser developer tools for WebSocket or polling errors.
- Check backend CORS and allowed frontend origin.
- Check that the frontend points to the intended backend URL.

14. Model Locks and Review Locks

STM Creator has two lock concepts.

14.1 REST Model Edit Lock

REST lock endpoints allow a model-level edit lock:

- Acquire lock.
- Renew lock.
- Release lock.
- Inspect current lock.

This is useful for workflows that need a coarse lock around a whole model.

14.2 Collaboration Entity Locks

Socket.IO lock events protect individual entities such as states or transitions during collaborative editing.

Examples:

- One user opens a state node for editing.
- The app acquires a lock for that state entity.
- Other users see the entity as locked.
- The lock is released when editing is complete or when the socket disconnects.

14.3 Admin Review Lock

Review lock is a stronger model-level restriction controlled by administrators.

When a model is under review lock:

- Editing and saving should be blocked.
- The lock may include a reason.
- Admins can apply or remove the lock.
- Users should wait until review is complete or ask an admin to release the lock.

15. Milestones and Version Comparison

Milestones are backend-saved snapshots of a model.

Typical actions:

- List milestones.
- Create a milestone before major edits.
- Open a milestone.
- Delete a milestone if permission allows it.
- Restore a milestone.

When restoring:

- The backend replaces the current model content with milestone content.
- Collaboration clients may receive a `model:restored` event.
- Users should refresh or reload the model after restore if their UI does not update automatically.

The frontend also contains local version comparison utilities. These are useful for visual comparison, but backend milestones are the persistent shared snapshot mechanism.

16. Sharing Model Permissions

Owners and administrators can manage model permissions.

Supported actions:

- View current permission entries.
- Grant a user `viewer`, `editor`, or `owner`.
- Update a user's model role.
- Remove a user's model permission.

Recommended permission practice:

- Grant the least role needed.
- Use `viewer` for reviewers who should not change the model.
- Use `editor` for collaborators who need to edit content.
- Reserve `owner` for people who can manage sharing and destructive operations.

Known implementation note:

- The backend permission list endpoint returns permission data from the model-permission service.
- If the frontend sharing modal appears empty despite backend data existing, compare the current frontend response expectation with the API response shape.

17. Model List

The model list modal helps users switch between available models.

The list may include:

- Owned models.
- Assigned/shared models.
- Templates.
- Admin-visible models.

If an expected model does not appear:

- Confirm the user is logged in.
- Confirm email verification.
- Confirm model-level permission exists.
- Ask an owner or admin to grant access.
- Check backend logs for query or permission errors.

18. Administrator Dashboard

The admin dashboard is available to global `Admin` users.

Admin functions include:

- View system statistics.
- List users.
- Change user global roles.
- Delete user sessions.
- Delete users where allowed.
- View audit log entries.
- Apply or release review locks through model flows.

Admin operations can affect other users. Before changing roles or deleting users, confirm the target account and the expected impact.

19. Import and Export

19.1 Import EKS JSON

Use File menu import to load an EKS-compatible JSON file.

After import:

- Inspect state names and IDs.

- Inspect transition source and target mapping.
- Inspect drivers and causal-chain fields.
- Save only after confirming the imported graph is correct.

Import does not guarantee that every historical or external JSON field maps perfectly to the current app schema.

19.2 Export EKS JSON

Export creates a JSON representation of the current model graph.

Use it for:

- Backup.
- Review.
- Transfer to compatible tooling.
- Debugging model data.

19.3 Export PNG

PNG export captures the current graph visual state.

Before exporting:

- Fit the graph into view.
- Hide panels that should not appear.
- Ensure labels are readable.

20. Troubleshooting

Login Succeeds but No Models Appear

Check:

- The account is verified.
- The account has model permissions.
- The backend is reachable.
- The model list endpoint is returning data.
- The frontend is using the intended API base URL.

Signup Works but User Cannot Create Models

Likely cause:

- The new user has global role `Viewer`.

Fix:

- An admin should promote the user to `Editor` if they need model creation rights.
- Or an owner/admin should grant model-level edit permission for a specific model.

Save Button Is Disabled or Save Fails

Check:

- Current model role.
- Global role.
- Review lock status.
- REST or socket lock status.
- Required model fields.
- Browser console network errors.
- Backend logs.
- Database connectivity.

Driver Search Has No Results

Check:

- Query length.
- Backend `/drivers/search` response.
- Driver data in the database.
- API base URL.

Comments Cannot Be Sent

Check:

- Login token.
- Model permission.
- Non-empty comment body.
- Comment panel target model name.
- Backend `/collab/:modelName/comments` response.

Frontend Uses Production Backend Instead of Local Backend

Check:

```
VITE_API_BASE_URL=http://localhost:3000
```

Then restart the Vite dev server. Vite environment variables are loaded at startup.

Backend Fails with `FRONTEND_URL required`

The backend environment validation requires the frontend URL.

Set:

```
FRONTEND_URL=http://localhost:5173
```

For production, set the DigitalOcean frontend origin:

```
FRONTEND_URL=https://stm-8nizc.ondigitalocean.app
```

Verification Email Is Not Sent

Check:

- SMTP or Gmail environment variables.
- Email service credentials.
- Backend logs.
- Whether the app is running in a mode that disables real email sending.
- Spam folder.

DigitalOcean Deployment Loads but API Calls Fail

Check:

- The frontend service environment variable used at build time.
- Backend service health.
- CORS `FRONTEND_URL`.
- Whether the browser request URL is the intended backend API.
- Whether DigitalOcean redeployed both services after environment changes.